Systolic Array for Effcient Matrix-Matrix Multiplication in Neural Network Implementations

Digital Hardware for Artificial Neural Network (ANN) Implementation

(Including Verilog Codes and Results)

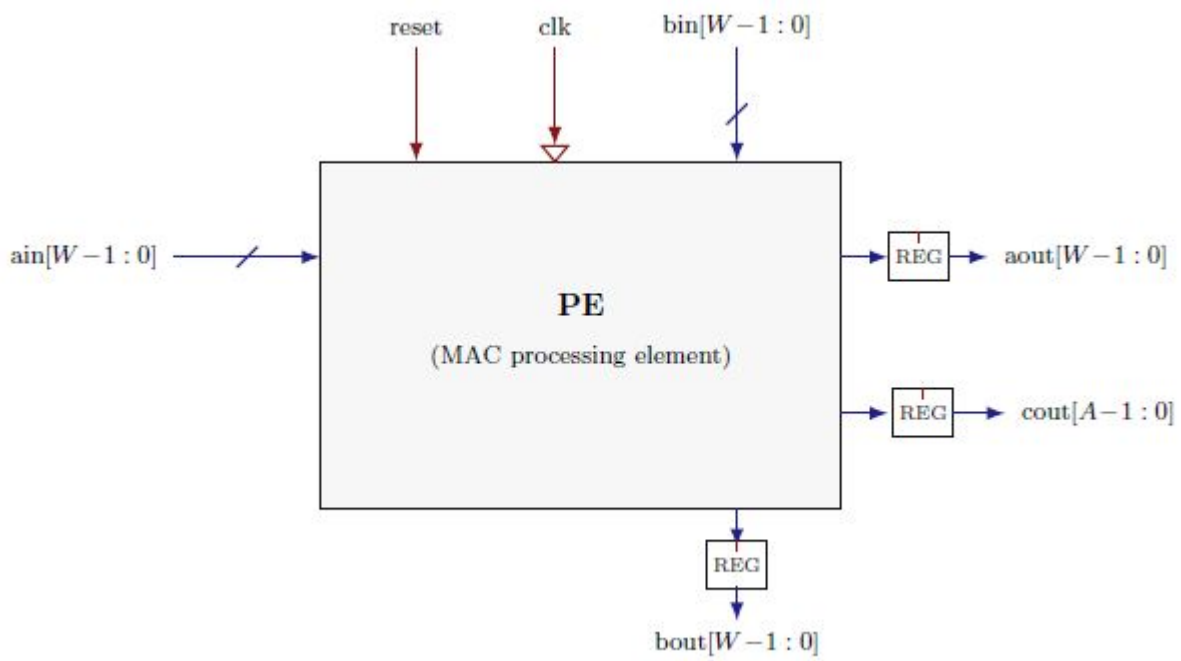
Prepared by:

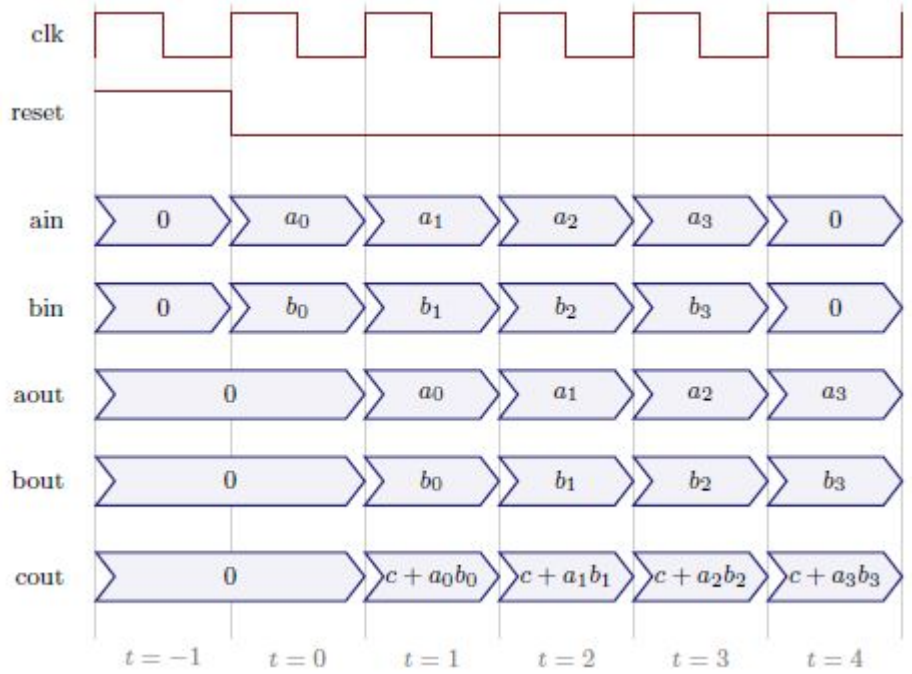
Anubhav Bhura, Debanjan Bhowmik

Department of Electrical Engineering

IIT Bombay

Processing Element (PE) for a Systolic Array





At a clock cycle:

$$\begin{aligned}\texttt{aout}(t) &= \texttt{ain}(t-1)\\ \texttt{bout}(t) &= \texttt{bin}(t-1)\\ \texttt{cout}(t) &= \texttt{cout}(t-1) + \texttt{ain}(t-1)\cdot\texttt{bin}(t-1)\end{aligned}$$

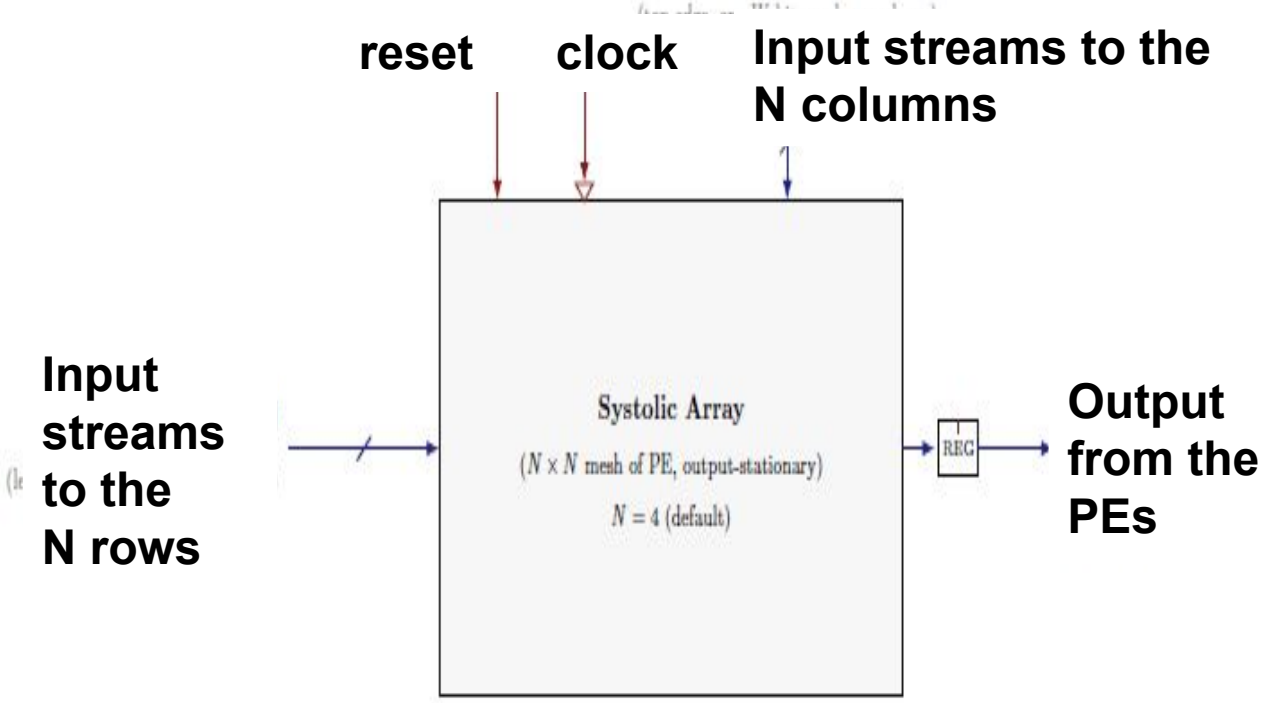
Over multiiple clock cycles:

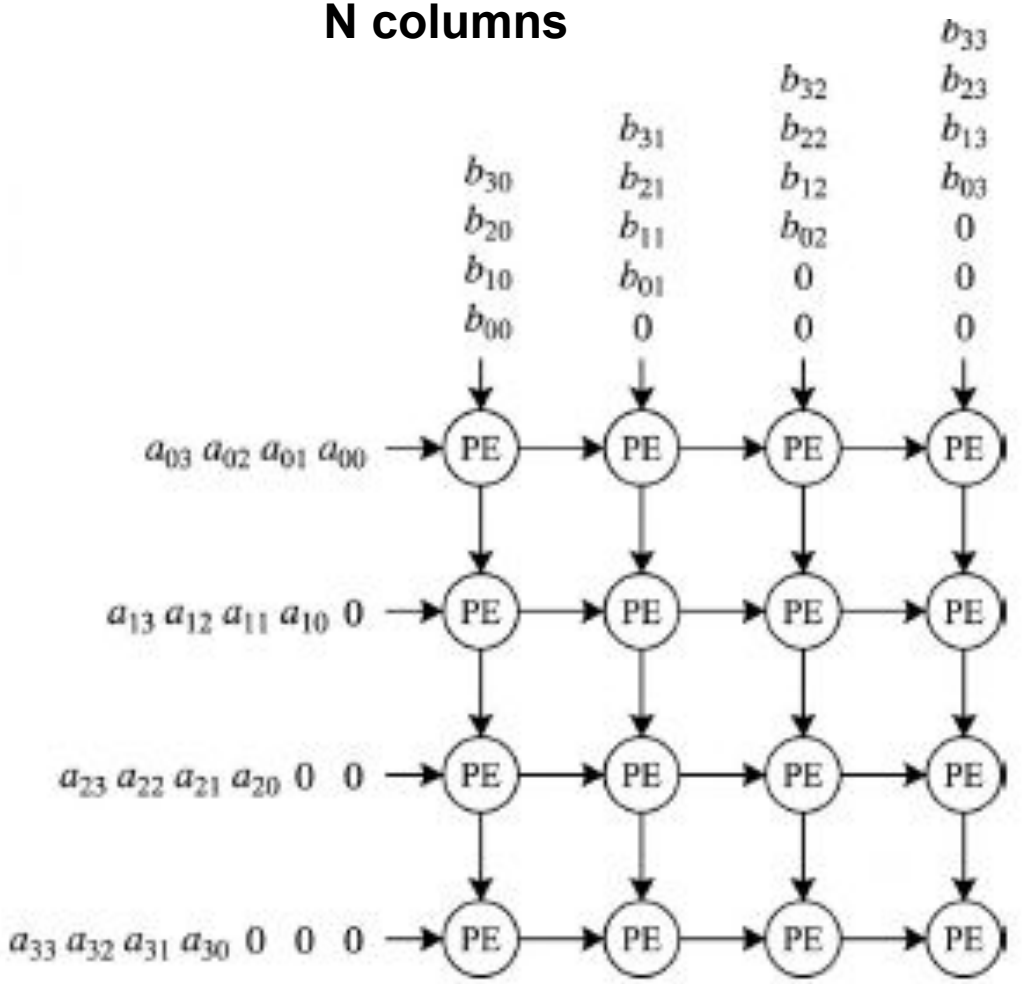
$$\texttt{cout}(t) = \sum_{k=0}^{t-1} \texttt{ain}(k)\cdot\texttt{bin}(k)$$

same as one row vector by one column vector multiplication

$$A_{1\times K}\, B_{K\times 1} = C_{1\times 1},$$

N x N Systolic Array: can multiply a NxK matrix (say A) and a KxN matrix (say B)

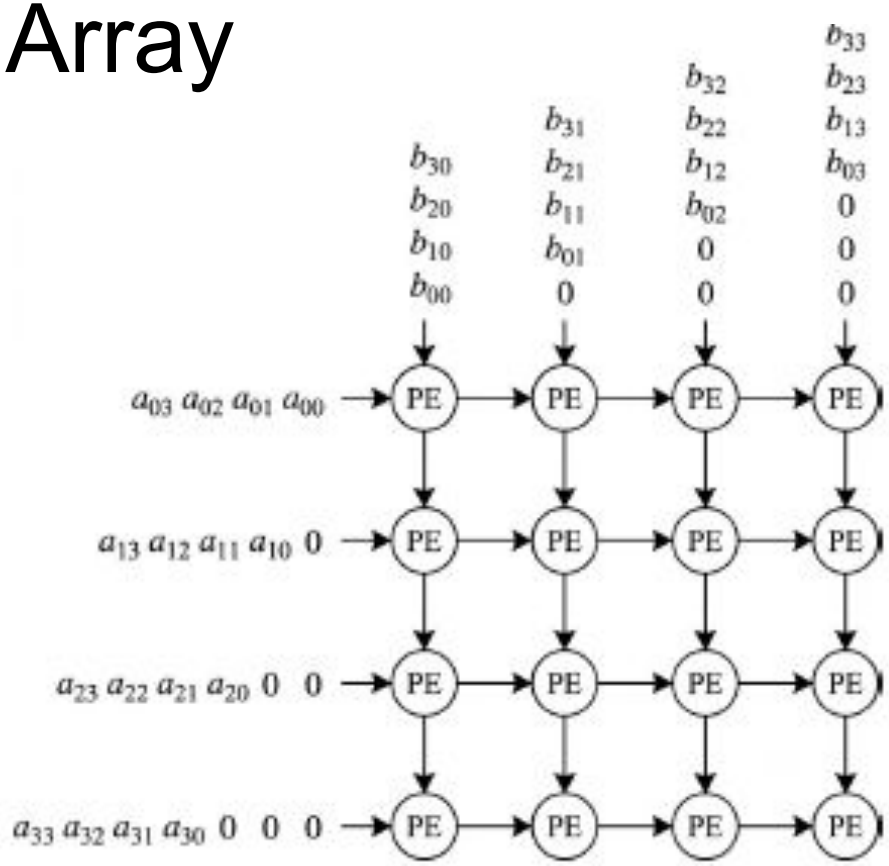




Total number of cycles required:

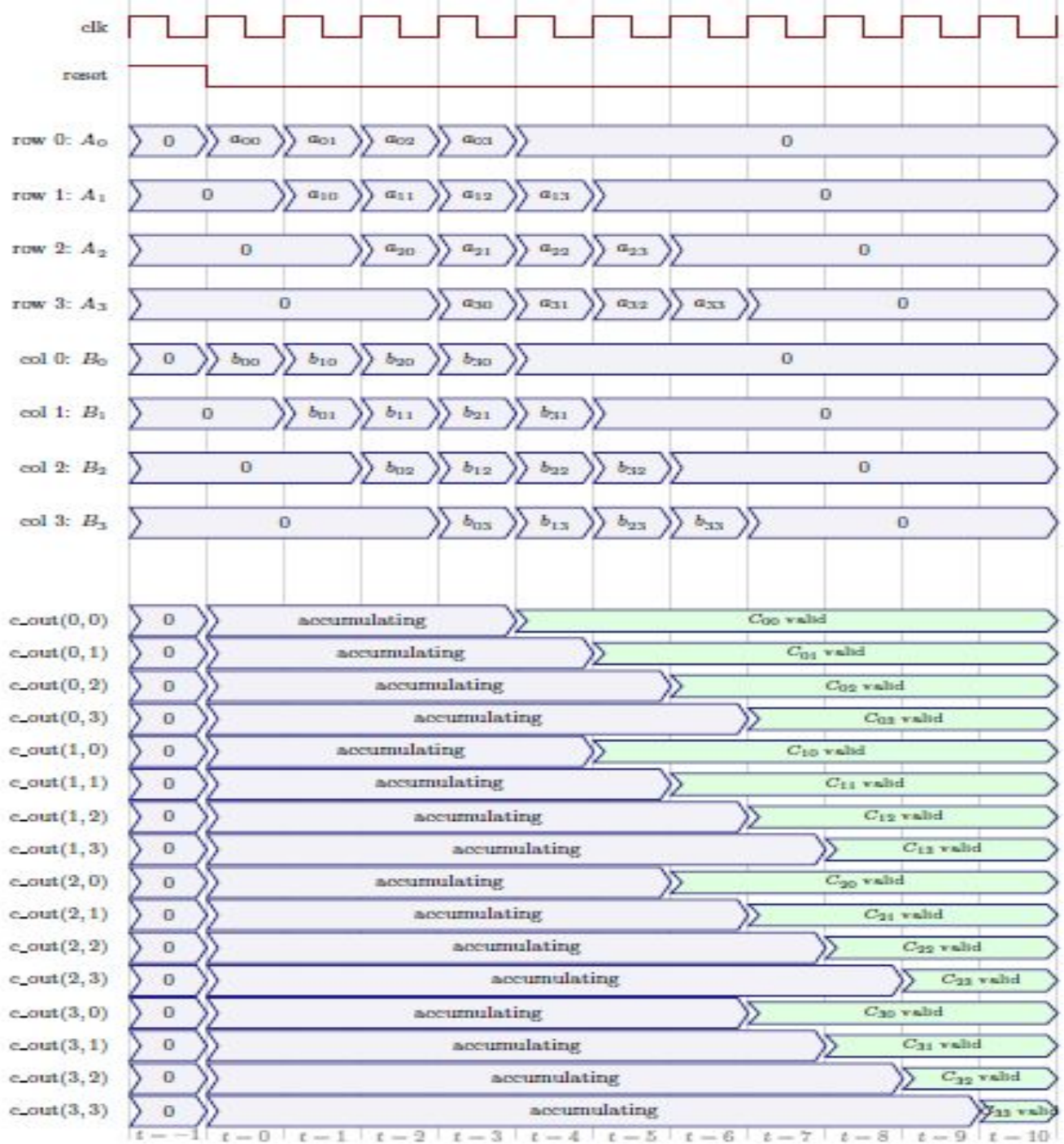
$$L = 2(N-1) + K = 2N + K - 2$$

N x N Systolic Array



Total number of cycles required:

$$L = 2(N-1) + K = 2N + K - 2$$



Verilog design code for Processing Element

$$\begin{aligned}\mathtt{aout}(t) &= \mathtt{ain}(t-1)\\ \mathtt{bout}(t) &= \mathtt{bin}(t-1)\\ \mathtt{cout}(t) &= \mathtt{cout}(t-1) + \mathtt{ain}(t-1)\cdot\mathtt{bin}(t-1)\end{aligned}$$

```
module PE(clk, rst, a_in, b_in, a_out, b_out, c_out);

parameter DATA_WIDTH = 16;
parameter ACC_WIDTH  = 32;


input wire clk, rst;
input wire signed [DATA_WIDTH-1:0] a_in, b_in;
output reg signed [DATA_WIDTH-1:0] a_out, b_out;
output reg signed [ACC_WIDTH-1:0]  c_out;


always @(posedge clk, posedge rst)
begin
      if (rst == 1'b1)
                  begin
         a_out <= 0;
         b_out <= 0;
         c_out <= 0;
      end
                  else
                  begin
         a_out <= a_in;
         b_out <= b_in;
         c_out <= c_out + (a_in * b_in);
      end
end

endmodule
```

Verilog design code for Systolic Array (Using Processing Element SubModule)

```
module SystolicArrayMod (Clk, Rst, A_row0, A_row1, A_row2, A_row3, B_col0, B_col1, B_col2,
B_col3,

C_00, C_01, C_02, C_03, C_10, C_11, C_12, C_13, C_20, C_21, C_22, C_23, C_30, C_31, C_32, C_33);

parameter DATA_WIDTH = 16;

parameter ACC_WIDTH  = 32;

parameter ROW = 4;  // Number of rows in the systolic array

parameter COL = 4;  // Number of columns in the systolic array

input wire Clk, Rst;

input wire signed [DATA_WIDTH-1:0] A_row0, A_row1, A_row2, A_row3, B_col0, B_col1, B_col2,
B_col3;

output wire signed [ACC_WIDTH-1:0] C_00, C_01, C_02, C_03, C_10, C_11, C_12, C_13, C_20, C_21,
C_22, C_23, C_30, C_31, C_32, C_33;
```

Verilog design code for Systolic Array (Using Processing Element SubModule)

```
wire signed [DATA_WIDTH-1:0] A[0:(ROW-1)][0:COL];

wire signed [DATA_WIDTH-1:0] B[0:ROW][0:(COL-1)];

wire signed [ACC_WIDTH-1:0] C[0:(ROW-1)][0:(COL-1)];

assign A[0][0] = A_row0;

assign A[1][0] = A_row1;

...

assign B[0][0] = B_col0;

assign B[0][1] = B_col1;

...;

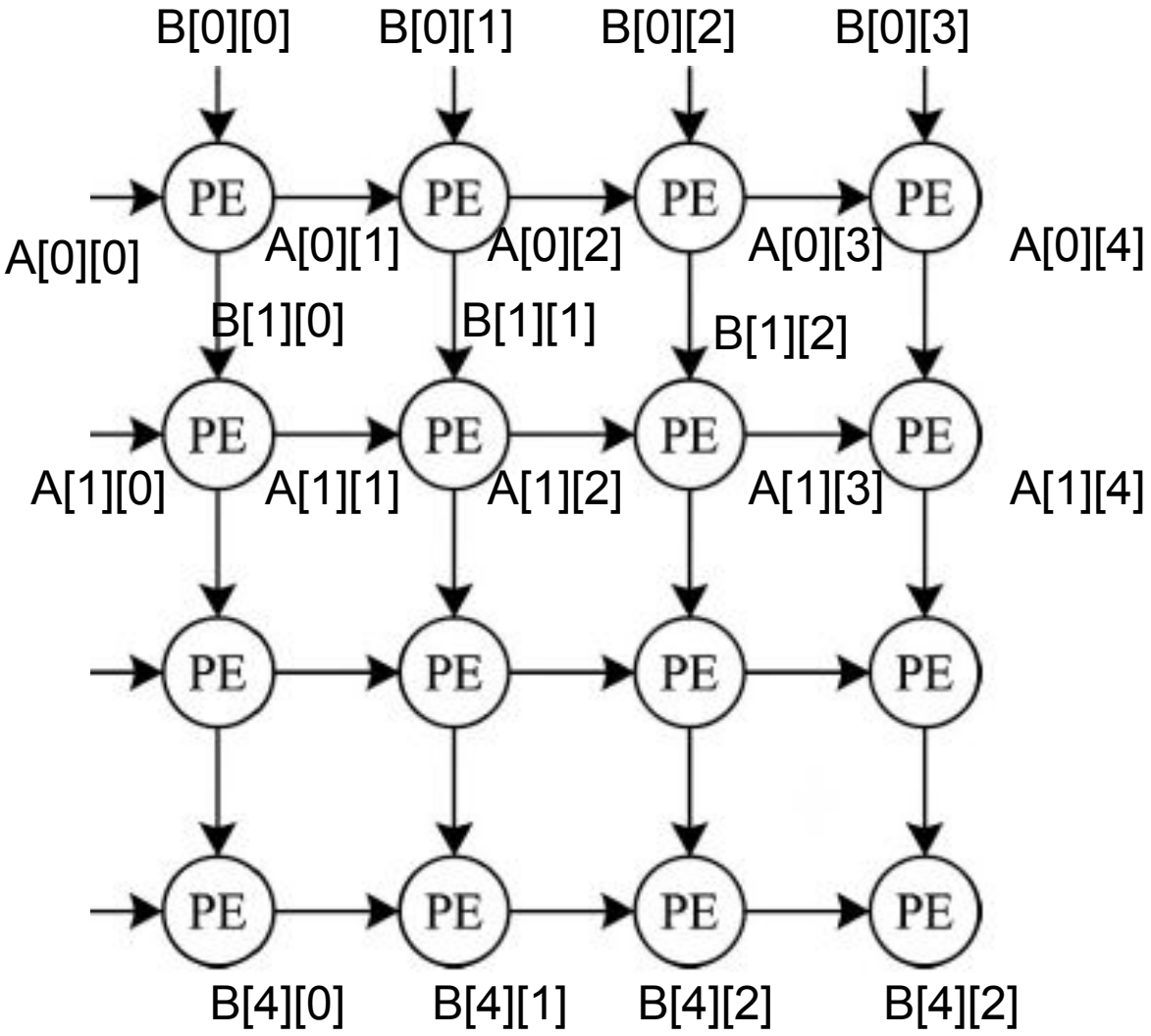
assign C_00 = C[0][0];

assign C_01 = C[0][1];

...
```

Verilog design code for Systolic Array (Using Processing Element SubModule)

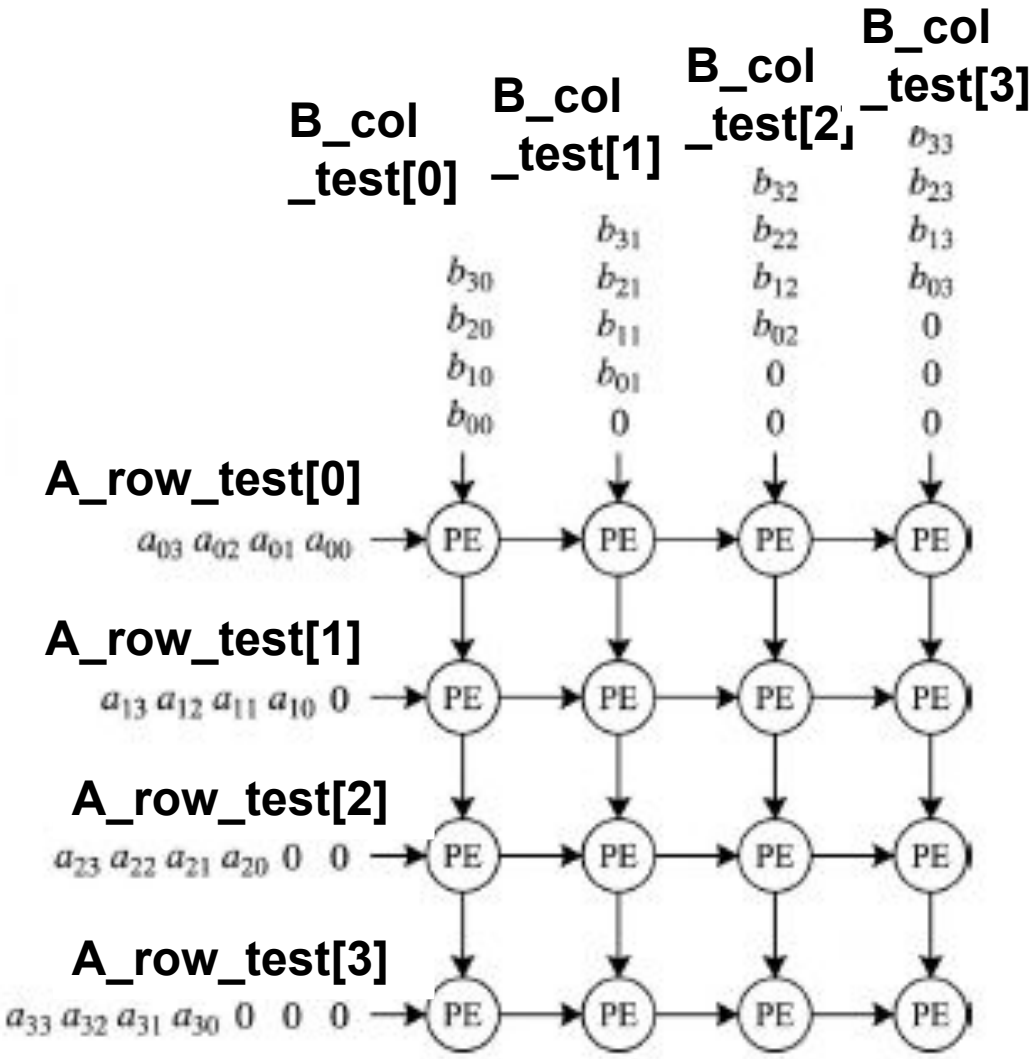
```
genvar r, c;
generate
for (r=0; r<ROW; r=r+1)
begin: genrow
        for (c=0; c<COL; c=c+1)
        begin: gencol
        PE PE_inst(
        .clk (Clk),
        .rst (Rst),
        .a_in (A[r][c]),
        .b_in (B[r][c]),
        .a_out(A[r][c+1]),
        .b_out(B[r+1][c]),
        .c_out(C[r][c]));
        end
end
endgenerate
endmodule
```



Verilog testbench code for Systolic Array (Multiplying ROW x Length matrix A and Length x COL matrix B), Main lines:

```
for (t = 0; t < ((2*(ROW-1))+ Length); t = t + 1)
        begin
    @(negedge clk_test);
begin
    for (r = 0; r < ROW; r = r + 1)
        begin
        col_idx = t - r;
    if (col_idx >= 0 && col_idx < COL)
            A_row_test[r] = A[r][col_idx];
                          else
 A_row_test[r] = {1*DATA_WIDTH{1'b0}};
    end

    for (c = 0; c < COL; c = c + 1)
begin
        row_idx = t - c;
        if (row_idx >= 0 && row_idx < ROW)
            B_col_test[c] = B[row_idx][c];
else
B_col_test[c] = {1*DATA_WIDTH{1'b0}};
    end
end
end
```

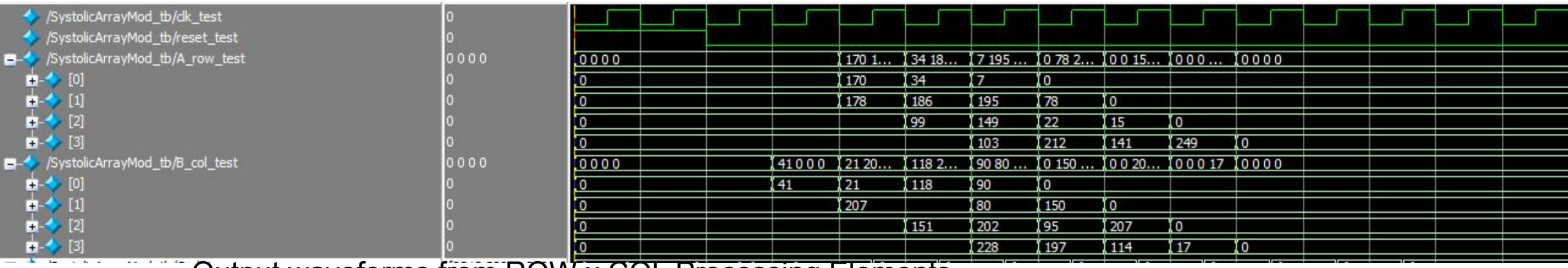


Verilog Behavioural Simulation Results

```
# ---- Matrix A ----
#   0 170  34   7
# 178 186 195  78
#  99 149  22  15
# 103 212 141 249
# ---- Matrix B ----
#  41 207 151 228
#  21 207 202 197
# 118  80  95 114
#  90 150 207  17
# ---- Comparing systolic array output to golden C = A x B ----
# match    C[0][0]: dut=8212 golden=8212
# match    C[0][1]: dut=38960 golden=38960
# match    C[0][2]: dut=39019 golden=39019
# match    C[0][3]: dut=37485 golden=37485
# match    C[1][0]: dut=41234 golden=41234
# match    C[1][1]: dut=102648 golden=102648
# match    C[1][2]: dut=99121 golden=99121
# match    C[1][3]: dut=100782 golden=100782
# match    C[2][0]: dut=11134 golden=11134
# match    C[2][1]: dut=55346 golden=55346
# match    C[2][2]: dut=50242 golden=50242
# match    C[2][3]: dut=54688 golden=54688
# match    C[3][0]: dut=47723 golden=47723
# match    C[3][1]: dut=113835 golden=113835
# match    C[3][2]: dut=123315 golden=123315
# match    C[3][3]: dut=85555 golden=85555
# *** PASS: all 16 elements match ***
#
```

Verilog Behavioural Simulation Results

Input waveforms: input to rows (A_row_test) and columns (B_col_test):



Output waveforms from ROW x COL Processing Elements

